

# Clase 07 — Repaso integrador: Fabric local con CouchDB

**Materia:** Desarrollo con Tecnologías Blockchain **Alumno:** Lucas Chas **Legajo:** 95346

**Fecha:** 08/10/2026

Todas las salidas de este documento son reales, de una ejecución de punta a punta. Los archivos completos están en la carpeta [evidencia/](#) y el chaincode en [inventario-ts/](#).

## Parte 1 — Versiones (teórico)

1. ¿Cuál es la línea LTS actual de Fabric (v2.5.x) y cuál es la última release publicada (v3.1.5)? ¿Qué aporta la nueva?

La línea LTS (soporte a largo plazo) es **v2.5.x**. En este TP se fijó la **v2.5.16** para peers, orderer y binarios, junto con **Fabric CA v1.5.22**. La última release publicada es la **v3.1.5**, de la línea 3.x. Lo principal que aporta la v3:

- **Ordering Service BFT (SmartBFT):** por primera vez el orderer tolera nodos maliciosos, no solo caídas.
- Soporte de firmas **Ed25519**, además de ECDSA.
- Limpieza de funciones heredadas: se elimina el *system channel* (los canales se crean con `osnadmin / channel participation API`) y el consenso Kafka/Solo.
- En la 3.1, mejoras de rendimiento para el chaincode, como escrituras y lecturas por lote (`StartWriteBatch/FinishWriteBatch`, `GetMultipleStates`).

Para la cursada conviene fijar la versión LTS: es la más estable y documentada, y así todos tenemos el mismo entorno, scripts e imágenes Docker que coinciden con la guía.

2. ¿Por qué el chaincode que escribiste para v2.5 corre igual en v3 sin cambios?

El chaincode no está atado a la versión del peer. Corre en un proceso o contenedor aparte y habla con el peer por el **protocolo gRPC de la chaincode shim API**, que se mantuvo compatible. Además, el contrato usa solo la API pública de `fabric-contract-api/fabric-shim` (`getState`, `putState`, `getQueryResult`, `getHistoryForKey`, `clientIdentity`), que no cambió. El ciclo de vida (*lifecycle* de 2.x: `package` → `install` → `approve` → `commit`) también es el mismo en v3. Los cambios de v3 están en el orderer y en la administración de la red, no en el contrato.

3. ¿Qué diferencia hay entre el consenso Raft y SmartBFT? ¿Cuándo justificás pasar a BFT?

|                        | Raft                                                          | SmartBFT                                                                                   |
|------------------------|---------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Tipo de falla tolerada | <b>CFT</b> ( <i>crash fault tolerant</i> ): nodos que se caen | <b>BFT</b> ( <i>byzantine</i> ): nodos que mienten, censuran o se comportan maliciosamente |
| Tolerancia             | f caídos con 2f+1 nodos                                       | f maliciosos con 3f+1 nodos                                                                |
| Modelo                 | Líder–seguidores, replicación de log                          | Varias rondas de votación entre nodos                                                      |
| Costo                  | Menos mensajes, menor latencia                                | Más mensajes y más latencia                                                                |

Pasar a BFT se justifica cuando **los nodos de ordering los operan organizaciones que no confían entre sí**. Por ejemplo, un consorcio donde cada miembro aporta un orderer y no se puede descartar que uno manipule el orden o

censure transacciones. Si todos los orderers los opera una sola entidad confiable, o el riesgo es solo de caídas, Raft alcanza y rinde mejor.

---

## 1. Entorno y versiones

---

```
$ docker --version
Docker version 29.6.2, build dfc4efb
$ docker compose version
Docker Compose version v5.3.1
$ git --version
git version 2.43.0
$ node --version
v22.22.0
$ npm --version
10.9.4
$ cat /etc/os-release | head -2
PRETTY_NAME="Ubuntu 24.04.4 LTS"
```

Fabric se instaló con el script oficial, con versiones fijas:

```
$ FABRIC_DOCKER_REGISTRY=docker.io/hyperledger ./install-fabric.sh --fabric-version 2.5.16 --ca-version 1.5.22 docker samples binary
...
hyperledger/fabric-peer:2.5.16
hyperledger/fabric-orderer:2.5.16
hyperledger/fabric-ccenv:2.5.16
hyperledger/fabric-baseos:2.5.16
hyperledger/fabric-ca:1.5.22
```

---

## 2. Red local con CouchDB

---

```
$ cd ~/fabric-labs/fabric-samples/test-network
$ ./network.sh down
$ ./network.sh up createChannel -c inventario -s couchdb
...
Channel 'inventario' created
...
Anchor peer set for org 'Org2MSP' on channel 'inventario'
Channel 'inventario' joined
```

```
$ docker ps --format "table {{.Names}}\t{{.Status}}"
NAMES                                STATUS
peer0.org1.example.com               Up 10 seconds
peer0.org2.example.com               Up 10 seconds
couchdb0                             Up 10 seconds
couchdb1                             Up 10 seconds
orderer.example.com                  Up 10 seconds
```

Qué representa cada contenedor CouchDB y por qué hay uno por peer: cada CouchDB es la **base de datos de estado (World State)** de un peer: couchdb0 es la de peer0.org1 (puerto 5984) y couchdb1 la de peer0.org2 (puerto 7984). Hay una por peer porque en Fabric **cada peer mantiene su propia copia** del ledger y del estado actual. No hay una base central compartida: cada organización tiene su copia y la reconstruye validando los mismos bloques. Con el flag `-s couchdb` se cambia el motor por defecto (LevelDB, embebido en el peer) por CouchDB. CouchDB guarda los valores como documentos JSON y permite **rich queries** sobre su contenido.

Evidencia extra: consultando CouchDB directamente se ve la base `inventario_inventario` (canal\_chaincode) y el documento JSON de un equipo dentro de `couchdb1`:

```
$ curl -s http://admin:adminpw@localhost:5984/_all_dbs # couchdb0 (peer0.org1)
["_replicator","_users","fabric__internal","inventario_","inventario_lifecycle", ... ,"inventario_inventario","inventario_lscc"]

$ curl -s http://admin:adminpw@localhost:7984/inventario_inventario/EQ-02 # couchdb1 (peer0.org2)
{"_id":"EQ-02","_rev":"1-b6838bca...","custodio":"Org1","descripcion":"Osciloscopio","docType":"equipo","estado":"disponible","id":"EQ-02","solicitante":"","ultimaAccionPor":"Org1MSP","ultimaActualizacion":"2026-10-08T12:13:33.239Z","~version":"CgMBBgA="}
```

### 3. Chaincode: máquina de estados y roles

Los cuatro archivos se crearon en `inventario-ts/` (`package.json`, `tsconfig.json`, `src/index.ts`, `src/inventario.ts`) con el contenido de la guía.

```
$ npm install
$ npm run build

> inventario@1.0.0 build
> tsc

compilación OK
```

Máquina de estados implementada:

```
disponible —SolicitarPrestamo (Org1)→ solicitado
solicitado —AprobarPrestamo (Org2)→ prestado
solicitado —RechazarPrestamo (Org2)→ disponible
prestado —RegistrarDevolucion (Org2)▶ disponible
```

Tres puntos del código:

1. **Doble control en cada operación (rol y estado).** Cada función que cambia el estado primero verifica **quién** llama, comparando `ctx.clientIdentity.getMSPID()` con `Org1MSP` u `Org2MSP`. Después verifica **en qué estado** está el equipo (`equipo.estado !== 'disponible', 'solicitado' o 'prestado'`). Si falla cualquiera de los dos, hace `throw new Error(...)`: la transacción no se endosa y no se escribe nada.
2. **getTxTimestamp() en lugar de Date.now().** El chaincode se ejecuta en **cada peer endosante por separado**. Con `Date.now()`, cada peer obtendría una hora distinta (por milisegundos), el *read-write set* resultante sería diferente y las firmas de endoso no coincidirían, así que la transacción se rechazaría. `getTxTimestamp()`

devuelve el timestamp que el **cliente puso en la propuesta**: es el mismo valor en todos los peers y hace que el resultado sea **determinista**.

3. La **rich query** usa `ctx.stub.getQueryResult(...)` con un selector JSON de CouchDB (`{ selector: { docType: 'equipo', estado } }`). Este método solo funciona con CouchDB como state database.

---

## 4. Despliegue

```
$ CHAINCODE_PATH="$HOME/fabric-labs/inventario-ts"
$ ./network.sh deployCC -c inventario -ccn inventario -ccp "$CHAINCODE_PATH" -ccl typescript -ccep
"AND('Org1MSP.peer','Org2MSP.peer')"
...
Chaincode is installed on peer0.org1
Chaincode is installed on peer0.org2
...
Committed chaincode definition for chaincode 'inventario' on channel 'inventario':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: t
rue, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'inventario'
```

Approvals: [Org1MSP: true, Org2MSP: true] indica que **las dos organizaciones aprobaron la definición** del chaincode antes del commit. La política `AND('Org1MSP.peer','Org2MSP.peer')` obliga a que toda transacción la firmen un peer de cada organización. Por eso la función `invocar` apunta a los dos peers (`--peerAddresses 7051 y 9051`).

---

## 5. Flujo entre las dos organizaciones

Se definieron las variables comunes y las funciones `usar_org` e `invocar` de la Parte 5 de la guía.

### Inicialización y consulta por clave

```
$ usar_org 1
Ahora sos Org1 (custodio)
$ invocar '{"function":"InitLedger","Args":[]}'
Chaincode invoke successful. result: status:200
$ peer chaincode query -C inventario -n inventario -c '{"Args":["ReadEquipo","EQ-01"]}'
{"custodio":"Org1","descripcion":"Servidor de pruebas","docType":"equipo","estado":"disponible","i
d":"EQ-01","solicitante":"","ultimaAccionPor":"Org1MSP","ultimaActualizacion":"2026-10-08T12:13:33.2
39Z"}
```

Los campos vuelven **ordenados alfabéticamente**: CouchDB guarda el JSON como documento y lo devuelve en su propio orden, no en el orden del código.

### Paso 1 — Org1 solicita el préstamo

```
$ usar_org 1
$ invocar '{"function":"SolicitarPrestamo","Args":["EQ-01"]}'
Chaincode invoke successful. result: status:200
$ sleep 3
```

```
$ peer chaincode query -C inventario -n inventario -c '{"Args":["ReadEquipo","EQ-01"]}'
{"custodio":"Org1","descripcion":"Servidor de pruebas","docType":"equipo","estado":"solicitado","id":"EQ-01","solicitante":"Org2","ultimaAccionPor":"Org1MSP","ultimaActualizacion":"2026-10-08T12:13:36.430Z"}
```

## Paso 2 — Org1 intenta actuar de nuevo y no puede

```
$ usar_org 1
$ invocar '{"function":"SolicitarPrestamo","Args":["EQ-01"]}'
Error: endorsement failure during invoke. response: status:500 message:"no se puede solicitar EQ-01: estado actual \"solicitado\". El turno es de Org2MSP"

$ invocar '{"function":"AprobarPrestamo","Args":["EQ-01"]}'
Error: endorsement failure during invoke. response: status:500 message:"solo la autoridad (Org2MSP) puede aprobar; cliente recibido: Org1MSP"
```

**Por qué Org1 quedó bloqueado:** después de solicitar, el turno pasó a Org2, y lo impidieron dos controles distintos:

- **Control por estado (re-solicitar):** Org1 sí tiene el rol para `SolicitarPrestamo`, pero esa operación exige estado de origen disponible y el equipo ya está solicitado. La máquina de estados no permite repetir la transición.
- **Control por rol (aprobar):** `AprobarPrestamo` exige que el cliente sea `Org2MSP`. La identidad de Org1, certificada por su MSP, no está autorizada, sin importar el estado. Org1 no puede aprobar su propia solicitud.

Los dos errores ocurren en la **fase de endoso**: el chaincode lanza la excepción, los peers no firman, la transacción nunca llega al orderer y el ledger no cambia.

## Paso 3 — Org2 aprueba

```
$ usar_org 2
Ahora sos Org2 (autoridad)
$ invocar '{"function":"AprobarPrestamo","Args":["EQ-01"]}'
Chaincode invoke successful. result: status:200
$ sleep 3
$ peer chaincode query -C inventario -n inventario -c '{"Args":["ReadEquipo","EQ-01"]}'
{"custodio":"Org1","descripcion":"Servidor de pruebas","docType":"equipo","estado":"prestado","id":"EQ-01","solicitante":"Org2","ultimaAccionPor":"Org2MSP","ultimaActualizacion":"2026-10-08T12:13:39.782Z"}
```

## Paso 4 — Org2 registra la devolución

```
$ usar_org 2
$ invocar '{"function":"RegistrarDevolucion","Args":["EQ-01"]}'
Chaincode invoke successful. result: status:200
$ sleep 3
$ peer chaincode query -C inventario -n inventario -c '{"Args":["ReadEquipo","EQ-01"]}'
{"custodio":"Org1","descripcion":"Servidor de pruebas","docType":"equipo","estado":"disponible","id":"EQ-01","solicitante":"","ultimaAccionPor":"Org2MSP","ultimaActualizacion":"2026-10-08T12:13:42.964Z"}
```

El ciclo se cerró: EQ-01 volvió a disponible y `ultimaAccionPor` registra que el último cambio lo hizo Org2.

---

## 6. Rich query sobre CouchDB

---

Antes de la nueva solicitud (los tres equipos disponibles):

```
$ peer chaincode query -C inventario -n inventario -c '{"Args":["EquiposPorEstado","disponible"]}'
[{"custodio":"Org1","descripcion":"Servidor de pruebas",...,"estado":"disponible","id":"EQ-01",...,"ultimaAccionPor":"Org2MSP",...},
{"custodio":"Org1","descripcion":"Osciloscopio",...,"estado":"disponible","id":"EQ-02",...},
{"custodio":"Org1","descripcion":"Impresora 3D",...,"estado":"disponible","id":"EQ-03",...}]
```

Org1 vuelve a solicitar EQ-01:

```
$ usar_org 1
$ invocar '{"function":"SolicitarPrestamo","Args":["EQ-01"]}'
Chaincode invoke successful. result: status:200
```

Después (solo EQ-02 y EQ-03):

```
$ peer chaincode query -C inventario -n inventario -c '{"Args":["EquiposPorEstado","disponible"]}'
[{"custodio":"Org1","descripcion":"Osciloscopio",...,"estado":"disponible","id":"EQ-02",...},
{"custodio":"Org1","descripcion":"Impresora 3D",...,"estado":"disponible","id":"EQ-03",...}]

$ peer chaincode query -C inventario -n inventario -c '{"Args":["EquiposPorEstado","solicitado"]}'
# extra
[{"custodio":"Org1","descripcion":"Servidor de pruebas",...,"estado":"solicitado","id":"EQ-01","solicitante":"Org2","ultimaAccionPor":"Org1MSP",...}]
```

(Salidas completas en `evidencia/07_rich_query.txt`.)

**Por qué cambió el resultado:** la rich query no busca por clave. Le pide a CouchDB todos los documentos cuyo **contenido** cumple `docType == "equipo"` y `estado == "disponible"`. Al commitearse la nueva solicitud, el documento de EQ-01 en el World State pasó a `estado: "solicitado"` y dejó de cumplir el selector.

**Con LevelDB**, que solo guarda pares clave → bytes y no entiende el JSON, no se puede filtrar por un campo. Habría que hacer una de dos cosas:

- recorrer todas las claves con `getStateByRange("", "")`, parsear cada valor y filtrar en el chaincode (lento y no escala), o
- mantener a mano un índice con **claves compuestas** (`createCompositeKey('estado~id', [estado, id])`), creándolo y borrándolo en cada cambio de estado, y consultarlo con `getStateByPartialCompositeKey`.

---

## 7. Historial del activo

---

```
$ peer chaincode query -C inventario -n inventario -c '{"Args":["HistorialEquipo","EQ-01"]}'
```

Resultado (de la más reciente a la más antigua, resumido; completo en `evidencia/08_historial.txt`):

| # | txId (abreviado)  | estado     | ultimaAccionPor | ultimaActualizacion | Operación         |
|---|-------------------|------------|-----------------|---------------------|-------------------|
| 5 | 03235d4b...acacde | solicitado | Org1MSP         | 12:13:46.310Z       | Nueva solicitud   |
| 4 | 3b7b4588...3dc7b  | disponible | Org2MSP         | 12:13:42.964Z       | Devolución        |
| 3 | a5dba9ad...6633d  | prestado   | Org2MSP         | 12:13:39.782Z       | Aprobación        |
| 2 | 839f5a49...bd17d  | solicitado | Org1MSP         | 12:13:36.430Z       | Solicitud         |
| 1 | 3c9fe9fb...14dc9  | disponible | Org1MSP         | 12:13:33.239Z       | Alta (InitLedger) |

Cada entrada tiene un `txId` distinto, el valor completo del activo en ese momento y qué organización hizo el cambio. Los dos intentos fallidos de Org1 del Paso 2 **no aparecen**: no pasaron el endoso y nunca se escribieron.

**De dónde sale el historial:** del **transaction log, la blockchain**, no del World State. El World State (CouchDB) solo tiene el valor **actual** de cada clave. `getHistoryForKey` usa el índice de historia del peer para ubicar, bloque por bloque, cada transacción válida que escribió esa clave.

**Por qué no se puede falsificar:** cada bloque contiene el hash del bloque anterior, así que modificar una transacción vieja rompe toda la cadena de hashes. Además, cada transacción va **firmada por el cliente** (su certificado X.509 emitido por el MSP de su organización) y por los **peers endosantes de Org1 y Org2**. Y como cada peer tiene su propia copia del ledger, una organización sola no puede reescribir su copia sin que deje de coincidir con la de la otra.

**Cómo queda registrado quién hizo cada cambio:** el contrato guarda en el propio activo `ultimaAccionPor = ctx.clientIdentity.getMSPID()`, que se ve en cada versión del historial. Más allá de ese campo, la transacción en el bloque incluye la **identidad firmante del creador**, que es la fuente criptográfica de esa autoría.

Detalle observado: la entrada #1 (InitLedger) tiene los campos en el orden del código, y las siguientes en orden alfabético. Es porque las demás operaciones leyeron el documento desde CouchDB (que lo devuelve ordenado), lo modificaron y lo volvieron a escribir tal cual.

## 8. Confianza: análisis

Las cinco capas que garantizan la confianza en Fabric, y dónde actuaron en este TP:

| Capa                                    | Qué garantiza                                                                                    | Dónde se vio en este TP                                                                                                                 |
|-----------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 1. Identidad (MSP / certificados X.509) | Se sabe quién es cada participante; red permitida                                                | <code>user_org</code> cambia de certificado ( <code>Admin@org1 / Admin@org2</code> ). El contrato lee <code>getMSPID()</code>           |
| 2. Lógica del contrato (chaincode)      | Las reglas de negocio se aplican igual para todos                                                | Doble control de rol y estado. Los dos errores de Org1 en el Paso 2                                                                     |
| 3. Política de endoso                   | Ninguna organización escribe sola                                                                | <code>AND('Org1MSP.peer', 'Org2MSP.peer')</code> , <code>Approvals: [Org1MSP: true, Org2MSP: true]</code> y los invokes a los dos peers |
| 4. Ordering y validación (consenso)     | Orden único y total de transacciones, y validación de firmas y versiones (MVCC) antes del commit | Orderer Raft ( <code>orderer.example.com</code> ). El estado cambia recién tras el commit, de ahí el <code>sleep 3</code>               |
| 5. Ledger inmutable e historial         | Lo escrito no se puede borrar ni alterar, y todo queda auditable                                 | <code>HistorialEquipo</code> con 5 <code>txId</code> distintos y quién hizo cada cambio                                                 |

---

## 9. Problemas encontrados

---

1. **Ruta del chaincode con espacios.** Al principio guardé el chaincode en la carpeta de mi repositorio, DTB - Lab CouchDB/inventario-ts. `./network.sh deployCC` partió la ruta en los espacios y corrió todos los parámetros (`CC_SRC_PATH: .../DTB`, `CC_VERSION: Lab`, `CC_SEQUENCE: CouchDB/inventario-ts`), y falló con `Path to chaincode does not exist` y `failed to read chaincode package at 'inventario.tar.gz'`. *Solución:* usar una ruta sin espacios, la que indica la guía (`~/fabric-labs/inventario-ts`). Log: `evidencia/09_problema_ruta_con_espacios.log`.
  2. **npm install falló dentro del contenedor de build del chaincode.** El peer construye el chaincode TypeScript en un contenedor `hyperledger/fabric-nodeenv:2.5`, que hace `npm install`. En mi entorno la salida a internet pasa por un proxy con certificado propio, y npm falló con `SELF_SIGNED_CERT_IN_CHAIN`. El peer lo reportó como `chaincode install failed with status: 500 ... string field contains invalid UTF-8`. *Solución:* crear una imagen local `fabric-nodeenv:2.5` derivada de la original que confía en ese certificado (`NODE_EXTRA_CA_CERTS / npm_config_cafile`) y repetir el `deployCC`. Es un problema del entorno de red, no del chaincode. Log: `evidencia/09_problema_npm_certificado.log`.
  3. **Latencia entre invoke y query.** El invoke devuelve `status:200` cuando se acepta la propuesta, pero el commit tarda uno o dos segundos. Por eso se espera con `sleep 3` antes de cada consulta, como recomienda la guía.
- 

## Parte 9 — Limpieza

---

```
$ cd ~/fabric-labs/fabric-samples/test-network
$ ./network.sh down
...
$ docker ps --format "{{.Names}}" | grep -E "peer|orderer|couchdb|inventario" || echo "red detenida OK"
red detenida OK
```

---

## 10. Respuestas finales

---

1. En el Paso 2, Org1 no pudo volver a actuar. ¿Qué dos controles distintos lo impidieron? ¿En qué se diferencian?

- **Control por estado:** al re-solicitar, Org1 tenía el rol correcto, pero `SolicitarPrestamo` exige estado `disponible` y el equipo estaba `solicitado`. Depende del **dato** guardado en el World State: si Org2 rechaza o se registra la devolución, Org1 vuelve a poder solicitar.
- **Control por rol:** al aprobar, `AprobarPrestamo` exige `Org2MSP` y el cliente era `Org1MSP`. Depende de la **identidad** del que firma: Org1 nunca va a poder aprobar, en ningún estado.

Uno responde a "¿es el momento correcto?" (máquina de estados) y el otro a "¿tenés permiso para hacer esto?" (autorización por MSP). Juntos impiden que una sola organización recorra todo el flujo por su cuenta.

2. ¿En qué se diferencia una consulta por clave (`ReadEquipo`) de una rich query (`EquiposPorEstado`)? ¿Cuál necesita CouchDB y por qué?

`ReadEquipo` usa `getState(id)`: busca **un** valor conociendo su clave exacta, y funciona igual con LevelDB y con CouchDB. `EquiposPorEstado` usa `getQueryResult(selector)`: busca **por el contenido** del valor (campos `docType` y `estado`) y puede devolver varios resultados sin conocer las claves. Esta necesita **CouchDB**, porque solo CouchDB guarda los valores como documentos JSON y entiende consultas con selectores (Mango queries). LevelDB guarda bytes opacos y solo sabe buscar por clave o por rango de claves.

### 3. ¿Por qué se usa `getTxTimestamp()` y no `Date.now()` dentro del chaincode?

Porque el chaincode se ejecuta por separado en cada peer endosante, y para que la transacción sea válida **todos tienen que producir exactamente el mismo resultado** (read-write set). `Date.now()` da la hora local de cada peer: el valor de `ultimaActualizacion` sería distinto en cada uno, los endosos no coincidirían y la transacción sería rechazada. `getTxTimestamp()` devuelve el timestamp que fijó el cliente en la propuesta, igual para todos los peers, y mantiene el chaincode **determinista**.

### 4. ¿De dónde sale el historial de un activo y por qué es confiable? ¿Cómo queda registrado qué organización hizo cada cambio?

Sale del **transaction log**, la **cadena de bloques**, no del World State, que solo tiene el valor actual. Es confiable porque los bloques están encadenados por hash (cualquier alteración se detecta), cada transacción está firmada por su creador y por los endosantes de las dos organizaciones, y cada organización tiene su propia copia del ledger. Qué organización hizo cada cambio queda registrado en dos lugares: en el dato (`ultimaAccionPor = getMSPID()`, visible en cada versión del historial) y, criptográficamente, en la firma y el certificado del creador que trae cada transacción del bloque.

### 5. Nombrá las cinco capas que garantizan la confianza en Fabric y dónde las viste actuar en este TP.

1. **Identidad (MSP)**: al cambiar entre `Admin@org1` y `Admin@org2` con `usar_org`, y en `getMSPID()` dentro del contrato.
2. **Lógica del chaincode**: los controles de rol y estado que bloquearon a `Org1` en el Paso 2.
3. **Política de endoso AND(0rg1, 0rg2)**: `Approvals: [0rg1MSP: true, 0rg2MSP: true]` y los invokes que necesitan las firmas de los dos peers.
4. **Ordering y validación**: el orderer Raft ordena las transacciones y los peers validan antes del commit. Por eso hay que esperar antes de ver el nuevo estado.
5. **Inmutabilidad e historial**: `HistorialEquipo` devolvió las 5 versiones de EQ-01 con su `txId` y su autor.

### 6. Para el caso de certificados académicos (Caso C del repaso): ¿qué campos tendría el activo "certificado", qué estados y qué rol tendría cada organización?

*Activo Certificado:*

| Campo                                | Descripción                                                                       |
|--------------------------------------|-----------------------------------------------------------------------------------|
| <code>docType</code>                 | "certificado" (para las rich queries)                                             |
| <code>id</code>                      | Número único de certificado                                                       |
| <code>alumnoId / alumnoNombre</code> | Legajo o DNI y nombre del egresado                                                |
| <code>carrera / titulo</code>        | Carrera y título otorgado                                                         |
| <code>fechaEgreso</code>             | Fecha de finalización                                                             |
| <code>hashDocumento</code>           | Hash (SHA-256) del PDF del diploma. El PDF no va en la blockchain, solo su huella |
| <code>emisor</code>                  | Facultad o universidad que lo emite                                               |

| Campo                                 | Descripción                                    |
|---------------------------------------|------------------------------------------------|
| estado                                | solicitado, emitido, rechazado o revocado      |
| motivo                                | Motivo de rechazo o revocación                 |
| ultimaActualizacion / ultimaAccionPor | Timestamp determinista y MSP del último cambio |

*Máquina de estados y roles:*

```
(nuevo)    —SolicitarEmision (Org1: Facultad)——> solicitado
solicitado —EmitirCertificado (Org2: Rectorado/Ministerio)——> emitido
solicitado —RechazarEmision (Org2)—————> rechazado
emitido    —RevocarCertificado (Org2)————> revocado
```

- **Org1 — Facultad (custodio de los datos académicos):** da de alta la solicitud con los datos del egresado y el hash del diploma. No puede emitir ni revocar.
- **Org2 — Rectorado o Ministerio (autoridad):** valida y emite, rechaza o revoca. No puede crear solicitudes.
- **Terceros verificadores (empresas, otras universidades):** solo consultan (`ReadCertificado`, `VerificarHash`, `HistorialCertificado`) para comprobar que un diploma es auténtico y no fue revocado.

Se usaría la misma política `AND(Org1, Org2)` y una rich query como `CertificadosPorEstado` o `CertificadosPorAlumno` sobre CouchDB.

## Lista final de revisión

- ☒ Levanté la red con `-s couchdb` (no la default con LevelDB).
- ☒ Vi los contenedores `couchdb0` y `couchdb1` en `docker ps`.
- ☒ Creé los cuatro archivos del chaincode (`package.json`, `tsconfig.json`, `src/index.ts`, `src/inventario.ts`).
- ☒ Creé el canal `inventario`.
- ☒ `npm run build` generó `dist/index.js`.
- ☒ Deploy con Approvals: `[Org1MSP: true, Org2MSP: true]`.
- ☒ Solicité el préstamo como Org1 (estado `solicitado`).
- ☒ Verifiqué que Org1 no puede re-solicitar (error por estado).
- ☒ Verifiqué que Org1 no puede aprobar (error por rol).
- ☒ Aprobé como Org2 (estado `prestado`) y registré la devolución.
- ☒ Ejecuté la rich query por estado antes y después de una solicitud.
- ☒ Recuperé el historial con varios `txId`.
- ☒ Ejecuté `./network.sh down`.